

Βιοπληροφορική

Εργασία 2026

Γεώργιος Νικολαΐδης – π21115

GitHub Repository: <https://github.com/Noveboi/Bioinformatics>

Εισαγωγή

Στην παρούσα εργασία ασχολούμαστε με την προσομοίωση ακολουθιών (αλληλουχιών) DNA, την εύρεση της καλύτερης δυνατής στοίχισης πολλαπλών ακολουθιών, και την εκπαίδευση ενός μοντέλου που αναγνωρίζει αν μια ξένη ακολουθία ανήκει (ή σχετίζεται ως ένα βαθμό) στο στοιχισμένο σύνολο. Η υλοποίηση για την ικανοποίηση των απαιτήσεων της εργασίας έγινε σε Python χωρίς καμία εξωτερική βιβλιοθήκη. Στο τελικό χρήστη διατίθεται διεπαφή τερματικού (command-line interface) με την οποία μπορεί πολύ εύκολα να εκτελέσει όλα τα βήματα που περιγράφονται στην εκφώνηση. Για περισσότερες πληροφορίες όσον αφορά τη διεπαφή, προβείτε στο ενότητα **“Παράρτημα”**.

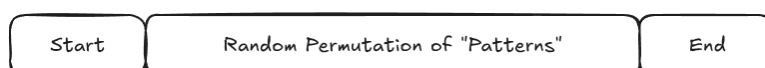
Περιεχόμενα

Εισαγωγή.....	2
I – Σύνθεση Ακολουθιών.....	4
II – Στοιχίση Ακολουθιών.....	6
Καθολική Στοιχίση (Global Sequence Alignment).....	6
Πολλαπλή Στοιχίση (Multiple Sequence Alignment - MSA).....	12
III – Προφίλ HMM.....	15
Θεωρία/Εισαγωγή στα HMM.....	15
Κατασκευή/Αρχικοποίηση.....	18
Εύρεση βέλτιστου μονοπατιού.....	21
Εκπαίδευση.....	24
IV – Βαθμολογία Στοιχίσης (Alignment Scoring).....	25
Δοκιμές και Αποτελέσματα.....	26
Παράρτημα.....	28

I – Σύνθεση Ακολουθιών

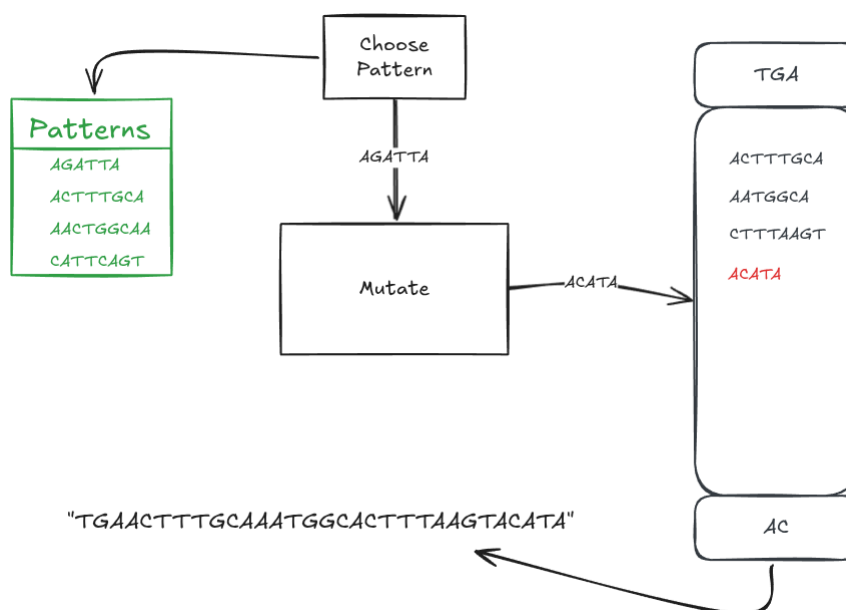
Το πρώτο μέρος της εκφώνησης πραγματεύεται την γέννηση ακολουθιών με βάση ένα αρχικό σύνολο μοτίβων (patterns). Η λέξη “γέννηση” χρησιμοποιήθηκε εσκεμμένα διότι οι ακολουθίες που παράγονται δεν είναι εντελώς τυχαίες και έχουν μια συγκεκριμένη δομή η οποία περιγράφεται και στην εκφώνηση.

Όπως φαίνεται στην παρακάτω εικόνα, κάθε παραγόμενη ακολουθία έχει μορφή με τρία μέρη:



- **Αρχή** (Start) – Ένα μικρό, τυχαίο πρόθεμα 1-3 συμβόλων
- **Κορμός** (Body) – Τυχαία αναδιάταξη των βασικών “patterns” όπου επιπλέον εισάγονται μικρές μεταλλάξεις έκαστος.
- **Τέλος** (End) – Ένα μικρό, τυχαίο επίθημα 1-3 συμβόλων

Πιο λεπτομερές, η διαδικασία αυτή απεικονίζεται ως εξής:



Αυτή η διαδικασία σύνθεσης αποσκοπεί στην παραγωγή ακολουθιών παρόμοιων μεταξύ τους, αλλά με μικρές διαφορές ώστε να προσομοιωθεί η μετάλλαξη του DNA. Επαναλαμβάνεται 200 φορές και χωρίζεται τυχαία σε τρία μέρη για να παραχθούν τρία σύνολα δεδομένων (datasets). Κάθε ένα από τα dataset έχει συγκεκριμένο ρόλο στα επακόλουθα μέρη. Αξίζει να τονίσουμε το γεγονός ότι και τα τρία dataset

προέρχονται από την ίδια πηγή (source) και μοιράζονται τα χαρακτηριστικά που αναφέρθηκαν παραπάνω.

II – Στοίχιση Ακολουθιών

Στο δεύτερο μέρος της εργασίας ασχολούμαστε με την ανάπτυξη δυο αλγορίθμων – ένα για καθολική στοίχιση δυο ακολουθιών, και ένα για την στοίχιση τριών ή περισσότερων ακολουθιών. Επειδή η διαδικασία πολλαπλής στοίχισης εξαρτάται από αυτή της καθολικής, θα ξεκινήσουμε με την ανάλυση της καθολικής.

Καθολική Στοίχιση (Global Sequence Alignment)

Η εκφώνηση περιγράφει μια διαδικασία καθολικής στοίχισης η οποία χρησιμοποιεί τη τεχνική του δυναμικού προγραμματισμού και υπολογίζει αποδοτικά αναδρομικό τύπο που εξαρτάται από τη σύγκριση δύο συμβόλων, ένα από κάθε ακολουθία. Ο αλγόριθμος που περιγράφεται εδώ είναι ο **Needleman-Wunsch**. Είσοδο δέχεται δύο ακολουθίες A, B με $|A| = N, |B| = M$. Υλοποιεί τον ακόλουθο αναδρομικό τύπο:

$$F(i, j) = \max \begin{cases} F(i-1, j-1) + s(A_i, B_j), \\ F(i-1, j) + d, \\ F(i, j-1) + d \end{cases}$$

όπου:

- $F(i, j)$ είναι το βέλτιστο σκορ στοίχισης για τις υποακολουθίες $A_1 A_2 \dots A_i$ και $B_1 B_2 \dots B_j$.
- $s(A_i, B_i)$ είναι το σκορ αντικατάστασης και εξαρτάται από το αν τα σύμβολα A_i και B_j είναι ίδια (τοπική ομοιότητα) ή όχι (τοπική ανομοιότητα).
- d είναι το gap penalty.

Η εκφώνηση μας δίνει τις τιμές των $s(A_i, B_j)$ και d :

$$s(A_i, B_j) = \begin{cases} 1 & , A_i = B_j, \\ -\frac{\alpha}{2} & , A_i \neq B_j \end{cases}$$

$$d = -\alpha$$

$$\alpha = 2 - \text{AM mod } 2$$

Τώρα που έχουμε ορίσει τον αναδρομικό τύπο, μπορούμε να κατασκευάσουμε το πίνακα δυναμικού προγραμματισμού που θα χρησιμοποιήσουμε. Για να δούμε το πίνακα, έστω $A = ATCG, B = ATGCA$, και $\alpha = 2$. Έχουμε:

		A	T	G	C	A
	0	-2	-4	-6	-8	-10
A	-2					
T	-4					
C	-6					
G	-8					

Πίνακας 1: Αρχικοποίηση πίνακα δυναμικού προγραμματισμού Needleman-Wunsch

Στο πίνακα 1 βλέπουμε τα σύμβολα της ακολουθίας A να είναι τοποθετημένα στις στήλες και τα σύμβολα της B στις γραμμές. Έτσι, κάθε κελί (i, j) του πίνακα απεικονίζει μια τιμή που θα εξάγεται επεξεργάζοντας τα σύμβολα A_i και B_j με τη χρήση του τύπου $F(i, j)$. Στο παραπάνω στιγμιότυπο του πίνακα έχουν ήδη τοποθετηθεί κάποιες τιμές ως μέρος της αρχικοποίησης του. Με βάση αυτές τις τιμές, θα υπολογιστούν οι επόμενες $F(i, j)$ για $i > 0, j > 0$. Για παράδειγμα, η τιμή $F(1, 1)$ υπολογίζεται με βάση τον ορισμό του τύπου:

$$F(1, 1) = \max \begin{cases} F(0, 0) + s(A_1, B_1), \\ F(0, 1) - 2, \\ F(1, 0) - 2 \end{cases}$$

$$F(1, 1) = \max \begin{cases} 0 + 1, \\ -2 - 2, \\ -2 - 2 \end{cases}$$

$$F(1, 1) = 1$$

Η διαδικασία αυτή επαναλαμβάνεται για όλα τα κελιά (i, j) με $0 < i \leq N, 0 < j \leq M$. Στην επόμενη σελίδα παρουσιάζονται μερικά στιγμιότυπα του DP (Dynamic Programming) πίνακα από την επαναλαμβανόμενη εκτέλεση της διαδικασίας αυτής.

		A	T	G	C	A
	0	-2	-4	-6	-8	-10
A	-2	1				
T	-4					
C	-6					
G	-8					

Πίνακας 2: Επανάληψη 1 ($i = 1, j = 1$)

		A	T	G	C	A
	0	-2	-4	-6	-8	-10
A	-2	1	-3			
T	-4					
C	-6					
G	-8					

Πίνακας 3: Επανάληψη 2 ($i = 1, j = 2$)

		A	T	G	C	A
	0	-2	-4	-6	-8	-10
A	-2	1	-3	-5		
T	-4					
C	-6					
G	-8					

Πίνακας 4: Επανάληψη 3 ($i = 1, j = 3$)

		A	T	G	C	A
	0	-2	-4	-6	-8	-10
A	-2	1	-1	-3	-5	-7
T	-4	-1	2	0	-2	-4
C	-6	-3	0	1	1	-1
G	-8	-5	-2	1	0	0

Πίνακας 5: Τελευταία επανάληψη ($i = N, j = M$)

Στον Πίνακα 5, βλέπουμε τα σκορ στοίχισης για κάθε υποακολουθία του A, B , καθώς και το σκορ στοίχισης για ολόκληρες τις ακολουθίες A, B στο κάτω δεξί κελί (όπου $i = N$ και $j = M$).

Για να βρούμε τη βέλτιστη καθολική στοίχιση, θα πρέπει να διατρέξουμε το πίνακα από κάτω προς τα πάνω. Για αυτή την διαδικασία της "στοίχισης", θα θεωρήσουμε δυο ακολουθίες $\hat{A}, \hat{B}$ οι οποίες θα είναι οι ανακατασκευασμένες ακολουθίες. Ο αλγόριθμος της στοίχισης θα ξεκινάει στο κελί (N, M) και κάθε φορά θα επιλέγει να πάει στο **πάνω**, **αριστερό** ή **διαγώνιο** κελί. Η απόφαση αυτή θα παρθεί με βάση τα υπολογισμένα σκορ – το κελί που θα επιλεγεί είναι αυτό που έχει το μέγιστο σκορ από τα τρία. Σε περίπτωση ισοπαλίας, στην δικιά μου υλοποίηση η σειρά προτεραιότητας/επιλογής είναι διαγώνια $\rightarrow$ πάνω $\rightarrow$ αριστερά. Η κάθε κίνηση αντιστοιχεί στην προσθήκη ενός συμβόλου στις ακολουθίες $\hat{A}, \hat{B}$.

1. Η **διαγώνια** κίνηση $(i, j) \rightarrow (i-1, j-1)$ αντιστοιχεί στις πράξεις $\hat{A} = \hat{A} \cup \{A_{i-1}\}$ και $\hat{B} = \hat{B} \cup \{B_{j-1}\}$. Είναι είτε μια τοπική ομοιότητα (match) είτε σε μια τοπική ανομοιότητα (mismatch)
2. Η κίνηση προς τα **πάνω** $(i, j) \rightarrow (i-1, j)$ αντιστοιχεί στις πράξεις $\hat{A} = \hat{A} \cup \{A_{i-1}\}$ και $\hat{B} = \hat{B} \cup \{-\}$. Βιολογικά, ερμηνεύεται ως μια **διαγραφή** (deletion) στην $\hat{B}$ και έτσι αναγράφεται και στην υλοποίηση του αλγόριθμου στη Python.
3. Η κίνηση προς τα **αριστερά** $(i, j) \rightarrow (i, j-1)$ αντιστοιχεί στις πράξεις $\hat{A} = \hat{A} \cup \{-\}$ και $\hat{B} = \hat{B} \cup \{B_{j-1}\}$. Βιολογικά, ερμηνεύεται ως μια **εισαγωγή** (insertion) στην $\hat{B}$, όπως και στην πάνω κίνηση, αυτός ο όρος χρησιμοποιείται στην υλοποίηση.

Στην υλοποίηση μου του Needleman-Wunsch, αυτές οι κινήσεις υπολογίζονται μαζί με τον υπολογισμό του πίνακα DP. Ένας επιπλέον πίνακας “trace” ορίζεται και αποθηκεύει τις βέλτιστες κινήσεις που πρέπει να κάνει κάποιος σε κάθε κελί. Ύστερα, μετά την κατασκευή του πίνακα “trace”, μια διαδικασία μπορεί να διατρέξει τον εν λόγω πίνακα και να παράξει τις ακολουθίες $\hat{A}$ και $\hat{B}$ όπως ακριβώς περιγράφηκε στη προηγούμενη παράγραφο.

```
# Main DP loop (Scoring Process)
for i in range(1, m + 1):
    for j in range(1, n + 1):
        match = dp[i - 1][j - 1] + s(seq1[i - 1], seq2[j - 1])
        delete = dp[i - 1][j] + gap_penalty
        insert = dp[i][j - 1] + gap_penalty

        best = max(match, delete, insert)

        dp[i][j] = best

        if best == match:
            trace[i][j] = Move.DIAGONAL
        elif best == delete:
            trace[i][j] = Move.UP
        elif best == insert:
            trace[i][j] = Move.LEFT
```

		A	T	G	C	A
	0	-2	-4	-6	-8	-10
A	-2	1	-1	-3	-5	-7
T	-4	-1	2	0	-2	-4
C	-6	-3	0	1	1	-1
G	-8	-5	-2	1	0	0

Πίνακας 6: Αναζήτηση καλύτερης διαδρομής (Επανάληψη 1) (Κόκκινο: Τρέχουσα βέλτιστη θέση, Πράσινο: Επόμενη βέλτιστη θέση, Κίτρινο: Υποψήφια θέση)

		A	T	G	C	A
	0	-2	-4	-6	-8	-10
A	-2	1	-1	-3	-5	-7
T	-4	-1	2	0	-2	-4
C	-6	-3	0	1	1	-1
G	-8	-5	-2	1	0	0

Πίνακας 7: Αναζήτηση καλύτερης διαδρομής (Επανάληψη 2)

		A	T	G	C	A
	0	-2	-4	-6	-8	-10
A	-2	1	-1	-3	-5	-7
T	-4	-1	2	0	-2	-4
C	-6	-3	0	1	1	-1
G	-8	-5	-2	1	0	0

Πίνακας 8: Αναζήτηση καλύτερης διαδρομής (Επανάληψη 3)

		A	T	G	C	A
	0	-2	-4	-6	-8	-10
A	-2	1	-1	-3	-5	-7
T	-4	-1	2	0	-2	-4
C	-6	-3	0	1	1	-1
G	-8	-5	-2	1	0	0

Πίνακας 9: Βέλτιστη διαδρομή

Πολλαπλή Στοίχιση (Multiple Sequence Alignment - MSA)

Η στοίχιση πολλών ακολουθιών είναι πιο σύνθετο θέμα από την καθολική στοίχιση, ωστόσο εξετάζοντας βήμα-βήμα τη διαδικασία θα δούμε ότι είναι απολύτως κατανοητή. Ο αλγόριθμος της πολλαπλής στοίχισης δέχεται είσοδο μια λίστα από ακολουθίες (θεωρείται ότι είναι πάνω από 2) και στόχο έχει την βέλτιστη στοίχιση τους σύμβολο-ανά-σύμβολο.

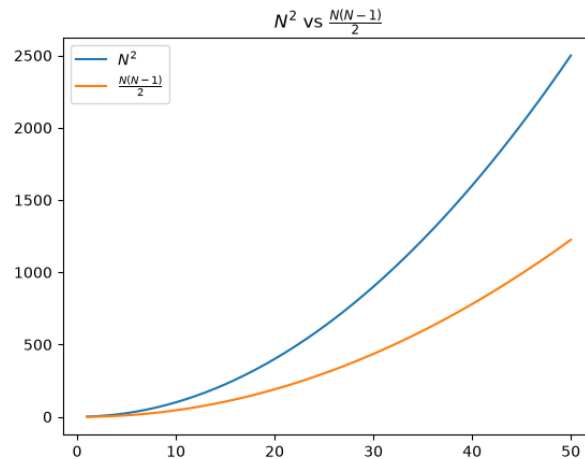
```
-GTAGATTAACTTTGCTAACTGGCAACATT-AGTGCG-
-CCCGATTAACTTTGCTAACTGGCAACATTCAGTTC--
CAGAGATTAACTTTGCAAACTGGCAGCATTTCAGT-C--
-GACGA-TAACTTTGCTAA-TGGCAACATTCAGT-CT-
--AAGATTAACTTTGC-AAAAGGCAACATT-AGTATG-
TG-AGATTAACTTTGCAACCTGGCAACATT--GT-TG-
C--ATATT-ACTTTGCAAACTGGCAGC-TTCAGTTCC-
-G-AGATTAACTTTGCAAACT-GCAACA-TCACTTCC-
-GGAGATTAACTTTG--AACTGGCCACATTCAGTT--
--TAAATTGACTTTGCAAACT-GC-ACATTCAGTGCC-
-TTAGA-TAAC-TTGTAACCTGGCAACATTCAGTAAG-
-G-AGA-TAAC-TTGCAACCTGGCAA-ATTCATTAAG-
-G-A-ATTAACCTTTGCAAACTGGC--CATT-AGTGGA-
--AGATT-AC-TTGCAAACTGGCAACATT--GTTC--
AAGAGATTAACTTTGC-ACCTGGCCAC-TTC-G-TCGC
-G-AGCTTAAC-TTGCAAACT-GCGACATTC-G-T-G-
-G-AGATCAACGTT-CAAACCTGCAACATTTAGTTC-A
--TAGCGTAACTTTGCACACT-GCAACATTCAG--CGG
C--AGATT-AC-TT-CAAACCTGGCAACA-TCA-AT-G-
AAAAGATTAACTTTG--AACCGGCAACA-TCAGTT--C
```

Εικόνα 1: Παράδειγμα εξόδου πολλαπλής στοίχισης. Παρατηρούμε πως τα περισσότερα κενά (-) είναι στα πρώτα/τελευταία 3 σύμβολα.

Ένας εξαντλητικός αλγόριθμος (brute-force / naive) θα χρειαζόταν να κατασκευάσει ένα N -διαστατό πίνακα DP και να κατασκευαστεί ένα υπερβολικά περίπλοκο γράφημα αναζήτησης καλύτερης διαδρομής. Γι' αυτό το λόγο επέλεξα άλλη προσέγγιση – την προοδευτική πολλαπλή στοίχιση ακολουθιών (progressive MSA), μια που ενδέχεται να μην δώσει τη βέλτιστη λύση, αλλά αρκετά καλή ώστε να θεωρηθεί αμελητέα η “ζημία” μιας μη-βέλτιστης στοίχισης. Ο αλγόριθμος θα αποτελείται από τα εξής βήματα:

1. Υπολογίζουμε τη καθολική στοίχιση $glob(x_i, x_j)$ (και το σκορ) για κάθε ζεύγος των N ακολουθιών $X = \{x_1, x_2, \dots, x_N\}$. Κατασκευάζουμε ένα δισδιάστατο

πίνακα $\mathbf{D} \in \mathbb{R}^{N \times N}$ όπου $D_{i,j} = glob(x_i, x_j)$. Ο πίνακας είναι συμμετρικός (symmetric matrix) επειδή ισχύει η ισότητα $glob(x_i, x_j) = glob(x_j, x_i)$. Λόγω της συμμετρικής ιδιότητας, μπορούμε έναντι των N^2 στοιχίσεων να υπολογίσουμε μόνο τις $\frac{N(N-1)}{2}$. Ο πίνακας $\mathbf{D}$ θα χρησιμοποιηθεί για την εύρεση των πιο όμοιων ζευγών ακολουθιών.



Εικόνα 2: Σύγκριση κλήσεων $glob(x_i, x_j)$. Σημειώνεται σημαντική μείωση πράξεων ακόμα και για μικρά N .

2. Από το πίνακα $\mathbf{D}$, βρίσκουμε το ζεύγος (x_i, x_j) με το μέγιστο σκορ. Το ζεύγος αυτό τοποθετείτε στη λίστα της πολλαπλής στοίχισης MSA. Στην συνέχεια επαναλαμβάνουμε τα παρακάτω βήματα έως ότου δεν υπάρχουν άλλες ακολουθίες για στοίχιση:
 1. Από τις ακολουθίες $\tilde{X}$ που απομένουν, επιλέγουμε το επόμενο καλύτερο ζεύγος $(\tilde{x}_i, \tilde{x}_j) \in \tilde{X}$. Από το ζεύγος, θα χρειαστούμε μόνο το $\tilde{x}_i$.
 2. Κατασκευάζουμε μια ακολουθία "consensus" c_k από την τρέχουσα MSA. Η ακολουθία "consensus" υπολογίζεται σύμβολο-ανά-σύμβολο συναρτήσσει των στηλών της MSA. Για κάθε στήλη, επιλέγεται το πιο κοινό σύμβολο (εκτός του κενού, το "consensus" δεν περιέχει κενά). Η consensus κατασκευάζεται μόνο και μόνο προκειμένου να μπορέσουμε να κάνουμε καθολική στοίχιση με την επόμενη καλύτερη ακολουθία του $\tilde{X}$. Αλλιώς, θα έπρεπε να κάνουμε πολλαπλή στοίχιση $|MSA| + 1$ ακολουθιών, κάτι που όπως αναφέρθηκε στην εισαγωγή της υπο-ενότητας, δεν είναι επιθυμητό – προτιμούμε να κάνουμε στοίχιση με 2 μόνο ακολουθίες.

x_1	A	T	G	C
x_2	A	-	C	G
x_3	A	-	G	C
c_k	A	T	G	C

Πίνακας 10: Παράδειγμα υπολογισμού consensus

3. Στοιχίζουμε καθολικά το consensus c_k με τη καλύτερη επόμενη ακολουθία $\tilde{x}_i$ -- $glob(c_k, \tilde{x}_i)$. Θεωρούμε της στοιχισμένες ακολουθίες $\hat{c}_k, \hat{\tilde{x}}_i$.
4. Προσθέτουμε (πιθανόν) κενά στην τρέχουσα λίστα ακολουθιών MSA βάσει της στοιχισμένης ακολουθίας $\hat{c}_k$. Αυτό γίνεται προκειμένου να τηρείτε ο κανόνας της πολλαπλής στοίχισης κατά τον οποίο πρέπει κάθε στοιχισμένη ακολουθία να είναι ίδιου μήκους. Για να καταλάβουμε το λόγο που χρειάζεται αυτό θεωρούμε το εξής παράδειγμα: $MSA = \{ACGT, A-GT\}$ και $c_k = ACGT$. Έστω ότι η επόμενη καλύτερη ακολουθία είναι η $x = ACAGT$. Η καθολική στοίχιση $glob(c_k, x)$ παράγει την ακολουθία $\hat{c}_k = AC-GT$. Οι ακολουθίες της MSA έχουν όλες μήκος 4, ενώ η $\hat{c}_k$ έχει 5. Εδώ γίνεται η εισαγωγή κενών στις ακολουθίες της MSA και γίνεται $MSA' = \{AC-GT, A--GT\}$. Τα κενά εισάγονται *πάντα* στις θέσεις όπου η στοιχισμένη consensus $\hat{c}_k$ έχει επίσης κενό ώστε να ελαχιστοποιηθεί το σφάλμα στοίχισης.
5. $MSA = MSA \cup \{\hat{\tilde{x}}_i\}$: Προσθέτουμε στην τρέχουσα λίστα πολλαπλών στοιχίσεων την στοιχισμένη (από το βήμα 2.3) ακολουθία $\hat{\tilde{x}}_i$.
6. $\tilde{X} = \tilde{X} - \{\tilde{x}_i\}$: Αφαιρούμε από τις απομένουσες ακολουθίες την ακολουθία $\tilde{x}_i$, η οποία στοιχίστηκε με το consensus, και εισήχθηκε στο σύνολο MSA.

Αυτή είναι πλήρης διαδικασία της προοδευτικής πολλαπλής στοίχισης ακολουθιών. Ουσιαστικά περιλαμβάνει την επαναληπτική εκτέλεση καθολικής στοίχισης μιας "σύνοψης" των τρεχόντων ακολουθιών της πολλαπλής στοίχισης (consensus) με την επόμενη πιο όμοια ακολουθία από αυτές που δεν έχουν εισαχθεί ακόμα στην πολλαπλή στοίχιση.

Το αποτέλεσμα της πολλαπλής στοίχισης (όπως φαίνεται στην Εικόνα 1) θα χρησιμοποιηθεί άμεσα στο επόμενο μέρος.

III – Προφίλ HMM

Στο τρίτο μέρος της εργασίας υλοποιούμε ένα συγκεκριμένο τύπο κρυφού μοντέλου Markov, ένα profile HMM. Ο σκοπός του profile HMM είναι να “μάθει” (μέσω εκπαίδευσης) τη δομή μιας οικογένειας ακολουθιών που έχουν κοινά χαρακτηριστικά, και να μπορεί να προσδιορίσει αν μια καινούργια, άγνωστη ακολουθία είναι πιθανό να ανήκει στην οικογένεια.

Θεωρία/Εισαγωγή στα HMM

Στην εμπειρία μου, τα HMM ήταν η πιο ξένη και δύσκολη έννοια για να κατανοήσω πλήρως, επομένως αφιερώνω μια υπο-ενότητα στην ανάπτυξη ενός HMM για να τα εμπεδώσω καλύτερα. Επίσης, είχα ένα πολύ καλό παράδειγμα στο νου! **Μπορείτε να προσπεράσετε αυτή την υπο-ενότητα.**

Τα κρυφά μοντέλα Markov λέγονται “κρυφά” επειδή ξέρουμε λίγα πράγματα για αυτό. Για παράδειγμα, έστω ότι έχουμε ένα φίλο στο Spotify, και βλέπουμε κάθε μέρα τι ακούει. Δεν ξέρουμε πώς νιώθει ο φίλος μας διότι δεν μιλάμε για τα συναισθήματά μας. Υποθέτουμε ότι ο φίλος έχει τρεις ψυχολογικές καταστάσεις: Ενθουσιασμένος, Αραχτός, ή του Ράγισαν τη Καρδιά. Αν μοντελοποιήσουμε τις ψυχολογικές καταστάσεις του φίλου μας ως “hidden states” και τα τραγούδια που ακούει ως “observations”, τότε βάσει τη θεωρία των HMM μπορούμε να συμπεράνουμε πιθανές ψυχολογικές καταστάσεις μόνο από τα τραγούδια που ακούει.

Ξέρουμε:

- Ότι όταν είναι “Ενθουσιασμένος”, ακούει κυρίως χαρούμενη και ελαφριά pop μουσική.
- Όταν είναι “Αραχτός”, ακούει ελαφριά prog-rock και nu-jazz.
- Όταν του “Ραγίσουν την Καρδιά”, ακούει metal.

Δεν ξέρουμε

- Ποτέ την ψυχολογική του κατάσταση.

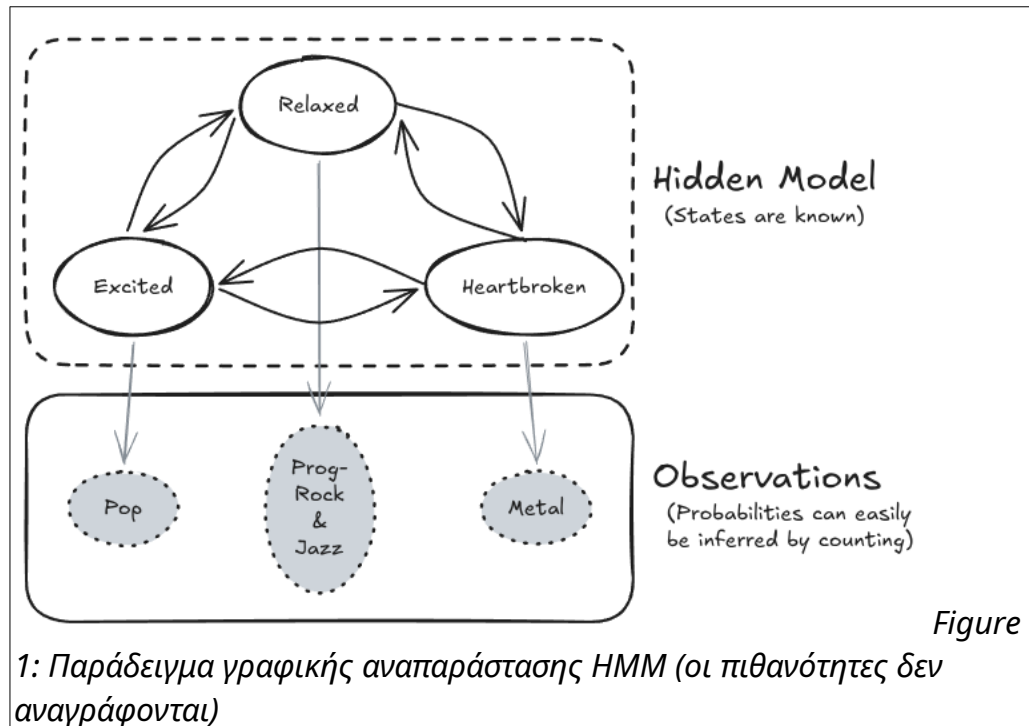
Έστω ότι για μια εβδομάδα, σημειώνουμε τα τραγούδια που ακούει:

Μέρα	Καλλιτέχνης	Τραγούδι	Γένος
1	Taylor Swift	Opalite	Pop
2	Sabrina Carpenter	Espresso	Pop
3	Dimmu Borgir	Puritania	Black Metal
4	Septicflesh	Great Mass of Death	Death Metal
5	Porcupine Tree	Shesmovedon	Progressive Rock
6	Kokoroko	Abusey Junction	Jazz Fusion
7	Rosalia	Berghain	Symphonic Pop

Πίνακας 11: Παρατηρήσεις (Observations)

Με βάση της παραπάνω παρατηρήσεις, μπορούμε να εξάγουμε πολλά συμπεράσματα για την ψυχολογική του κατάσταση, άλλα κάποια είναι πιο πιθανά από άλλα. Για παράδειγμα, μπορούμε με μεγάλη σιγουριά να πούμε ότι τη Μέρα #3 του ράγισαν τη καρδιά, και τη Μέρα #5 ηρέμησε και έγινε αραχτός πάλι (τα ξεπερνάει γρήγορα!).

Αν φανταστούμε ότι έχουμε παρατηρήσεις ενός χρόνου (δηλαδή 365 παρατηρήσεις), μπορούμε με αρκετή αυτοπεποίθηση να φτιάξουμε μια κατανομή πιθανοτήτων για τις μεταβάσεις μεταξύ των ψυχολογικών καταστάσεων για να μοντελοποιήσουμε και να μπορούμε να προβλέψουμε την μελλοντική ψυχολογική κατάσταση του φίλου μας.



Για το πρόβλημα που αντιμετωπίζουμε στην εργασία βιοπληροφορικής, θέλουμε να φτιάξουμε ένα μοντέλο το οποίο ενθυλακώνει τις στατιστικές ιδιότητες μιας οικογένειας ακολουθιών DNA.

Κατασκευή/Αρχικοποίηση

Το profile HMM έχει τρεις βασικές καταστάσεις: MATCH, INSERT, και DELETE, όλες με κρυφές πιθανότητες μετάβασης τις οποίες θέλουμε να βρούμε/εκτιμήσουμε. Ένα profile HMM κατασκευάζεται αρχικά από το αποτέλεσμα μιας πολλαπλής στοίχισης ακολουθιών. Η πολλαπλή στοίχιση MSA μας δίνει μια καλή εκτίμηση για τις θέσεις των ακολουθιών που είναι οι πιο **ομογενείς** και ταυτόχρονα βλέπουμε και ποιες είναι πιο “τυχαίες” ή μεταλλαγμένες. Για παράδειγμα, έστω η παρακάτω πολλαπλή στοίχιση τεσσάρων ακολουθιών:

	1	2	3	4	5
x_1	-	G	C	A	T
x_2	-	G	G	A	T
x_3	C	G	C	-	A
x_4	-	G	C	A	-

Με βάση την πολλαπλή στοίχιση, μπορούμε να συμπεράνουμε ότι:

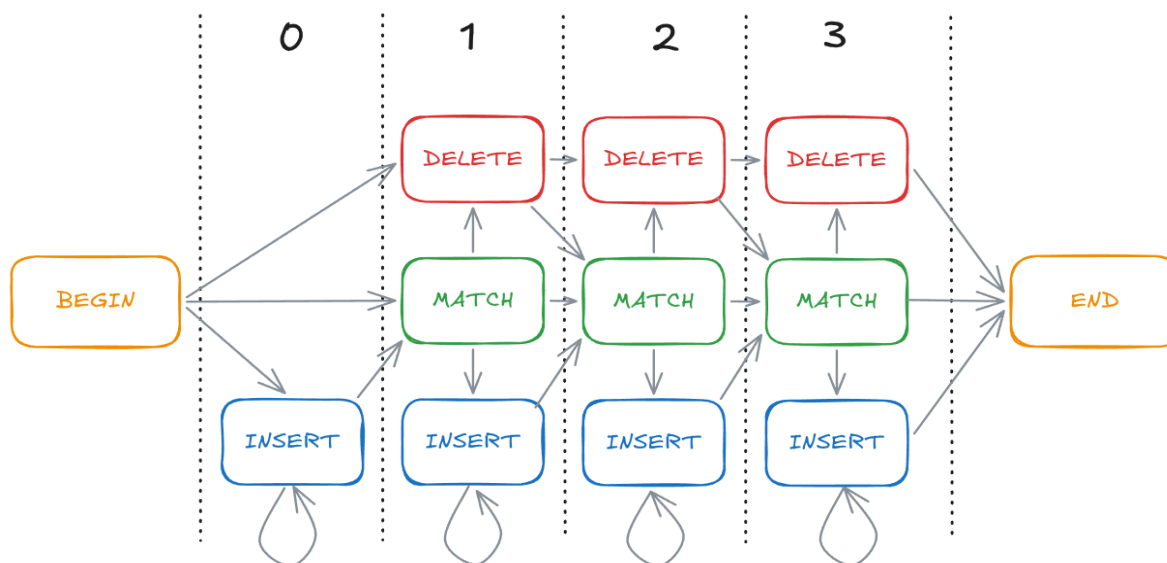
1. Η στήλη/θέση 1 είναι κατά πολύ μεγάλο ποσοστό άσχετη.
2. Η στήλη/θέση 2 είναι κατά πολύ μεγάλο βαθμό ομογενής
3. Η στήλη/θέση 5 είναι λιγότερο ομογενής από τη θέση 2, αλλά έχει ακόμα ένα βαθμό ομογένειας.

Η δουλειά του profile HMM είναι να αναλύσει το MSA στήλη-ανά-στήλη και για καθεμιά να αποφασίσει αν είναι ομογενής. Η απάντηση στο “είναι ομογενής;” για μια στήλη εξαρτάται από ένα κατώφλι (threshold) που ορίζουμε εμείς (είναι δηλαδή υπέρ-παράμετρος – hyperparameter). Ο αλγόριθμος απόφασης άρα είναι απλά η ερώτηση “Για την στήλη k , είναι το ποσοστό των κενών μικρότερο από το κατώφλι;”. Αν η απάντηση είναι ΝΑΙ, τότε η στήλη είναι ομογενής, αλλιώς δεν είναι. Έστω ότι θέτουμε το κατώφλι στο 50%, για την πολλαπλή στοίχιση του προηγούμενου πίνακα, έχουμε:

	1	2	3	4	5
x_1	-	G	C	A	T
x_2	-	G	G	A	-
x_3	C	G	C	-	A
x_4	-	G	C	A	-
% of gaps	75%	0%	0%	25%	50%
Homogeneous?	NO	YES	YES	YES	NO

Πίνακας 12: Αλγόριθμος απόφασης ομογένειας ανά στήλη

Οι ομογενείς στήλες μιας πολλαπλής στοίχισης στο κώδικα αναφέρονται ως “MATCH Columns” και είναι πολύ σημαντικές για την κατασκευή του profile HMM. Με βάση αυτές, υπολογίζονται οι αρχικές κατανομές πιθανοτήτων εκπομπής (emission probability distribution) για τις καταστάσεις INSERT και MATCH. Προτού δούμε αναλυτικά τους ορισμούς και τις ιδιότητες των πιθανοτικών κατανομών, αξίζει να δούμε μια γραφική αναπαράσταση του profile HMM:



Εικόνα 3: Profile HMM -- Γράφημα καταστάσεων. Δεν αποτυπώνονται όλες οι ακμές μεταξύ των καταστάσεων για λόγους διαύγειας στην εικόνα.

Βλέπουμε ότι υπάρχουν πολλαπλές βασικές καταστάσεις INSERT, DELETE, και MATCH. Αυτό γιατί κάθε στήλη/θέση μιας ακολουθίας θα αναπαρίσταται από μια τριάδα καταστάσεων INSERT, MATCH, και DELETE. Τα νούμερα στο πάνω-πάνω της Εικόνας 3

είναι ο δείκτης θέσης/στήλης. Το γράφημα παραλείπει το τι εκπέμπει κάθε κατάσταση.

Η κατάσταση MATCH έχει μοναδική κατανομή πιθανοτήτων εκπομπής $P_{M_i}(S)$ για κάθε **ομογενής** στήλη i . Η κατάσταση INSERT έχει μια καθολική κατανομή πιθανοτήτων $P_I(S)$. Ισχύει $S = \{A, T, G, C\}$. Τέλος, η κατάσταση DELETE δεν ασχολείται με πιθανότητες επειδή το εκπέμπει μόνο το κενό σύμβολο “-”.

Οι αρχικές κατανομές πιθανοτήτων των INSERT και MATCH υπολογίζονται με απλό μέτρημα των συμβόλων σε συνδυασμό με τη μέθοδο της **προσθετικής εξομάλυνσης** (additive/Laplace smoothing). Έστω X ένα σύνολο με πληθάριθμο N . Επίσης ορίζεται η συνάρτηση $l(X = s)$ η οποία μετράει πόσα σύμβολα s υπάρχουν στο σύνολο X . Η κλασική μέθοδος μετατροπής μετρήσεων σε πιθανότητες είναι η διαίρεση του αριθμού κάποιου συμβόλου με το συνολικό αριθμό συμβόλων στη στήλη:

$$P(S = s) = \frac{l(X = s)}{N} (1)$$

Για σύμβολα όμως που δεν υπάρχουν στο σύνολο, η πιθανότητα θα ήταν 0. Επειδή, λόγω της φύσης της εργασίας, δουλεύουμε με μικρά σύνολα δεδομένων, συμφέρει να δώσουμε μια πολύ μικρή πιθανότητα ακόμα και στα σύμβολα που δεν εμφανίζονται. Αυτό μπορούμε να το επιδιώξουμε μέσω της προσθετικής εξομάλυνσης, θεωρώντας μια σταθερά “ψευδομέτρησης” α (pseudo-count). Τροποποιούμε την εξίσωση (1) ως εξής:

$$P(S = s) = \frac{l(X = s) + \alpha}{N + \alpha d},$$

όπου $d = |S|$. Η παραπάνω εξίσωση εφαρμόζεται για κάθε ομογενής στήλη j για να υπολογίσουμε τις επιμέρους κατανομές MATCH $P_{M_i}(S)$. Επίσης εφαρμόζεται καθολικά για όλες τις μη-ομογενείς στήλες για να υπολογιστεί η κατανομή INSERT $P_I(S)$.

Τέλος, αρχικοποιούμε και τις κατανομές πιθανοτήτων μεταβάσεων (transition probabilities) ανά στήλη i . Αρχικά, η κάθε κατανομή θα υπολογίζεται από μια απλή μέτρηση των στηλών της MSA. Στην *εκπαίδευση*, θα βρεθούν οι “κρυμμένες” κατανομές πιθανοτήτων μεταβάσεων.

Εύρεση βέλτιστου μονοπατιού

Σε αυτή την υπο-ενότητα, θα αναλύσουμε πώς μπορούμε να βρούμε την βέλτιστη ακολουθία καταστάσεων (INSERT, MATCH, DELETE) για μια δοσμένη ακολουθία x . Εδώ παρατηρούμε εύκολα την που έχει αυτή η διαδικασία με τις ιδιότητες των HMM. Αν θεωρήσουμε ότι κάθε σύμβολο της ακολουθίας x είναι μια παρατήρηση (observation) και INSERT, MATCH, DELETE είναι οι κρυφές καταστάσεις, τότε ουσιαστικά κάνουμε inference (συμπερασμό) πάνω στο εν λόγω κρυφό μοντέλο Markov. Η *εύρεση βέλτιστου μονοπατιού* που αναφέρεται στην κεφαλίδα της παρούσας υπο-ενότητας ισοδύναμα λέγεται **εύρεση της πιο πιθανής ακολουθίας καταστάσεων**. Ο αλγόριθμος **Viterbi** μας δίνει τη “συνταγή” για την αναφερόμενη εύρεση.

Όπως και στην καθολική στοίχιση, ο αλγόριθμος Viterbi υλοποιείται χρησιμοποιώντας την τεχνική του δυναμικού προγραμματισμού. Εδώ τα πράγματα είναι λίγο πιο σύνθετα, καθώς ο πίνακας DP έχει 3 διαστάσεις και ορίζεται ως $V \in (0, 1]^{L \times N \times 3}$, όπου:

- L : Το πλήθος των ομογενών στηλών (match columns)
- N : Το πλήθος συμβόλων της κάθε ακολουθίας της πολλαπλής στοίχισης MSA
- 3 : Το πλήθος των καταστάσεων INSERT, MATCH, και DELETE

Ο αναδρομικός τύπος είναι (απλοποιημένα) ο εξής:

$V_s(i, j)$ = η μέγιστη πιθανότητα ενός μονοπατιού να καταλήξει στην i -οστή κατάσταση s μετά από την επεξεργασία j συμβόλων.

Ο υπολογισμός (και επομένως ο τύπος) του $V_s(i, j)$ εξαρτάται από την κατάσταση s , διότι όπως είδαμε και στην Εικόνα 3, οι καταστάσεις του profile HMM δεν είναι όλες συνδεδεμένες μεταξύ τους. Συγκεκριμένα, δεν υπάρχουν κύκλοι (έκτος από τις αυτό-αναφορές). Ας αναλύσουμε τους τύπου ένα-ένα για κάθε κατάσταση:

$$V_M(i, j) = \log P_{M_i}(S = x_j) + \max \begin{cases} V_M(i-1, j-1) & + \log P(M_{i-1} \rightarrow M_i) \\ V_I(i-1, j-1) & + \log P(I_{i-1} \rightarrow M_i) \\ V_D(i-1, j-1) & + \log P(D_{i-1} \rightarrow M_i) \end{cases}$$

$$V_I(i, j) = \log P_I(S = x_j) + \max \begin{cases} V_M(i, j-1) + \log P(M_i \rightarrow I_i) \\ V_I(i, j-1) + \log P(I_i \rightarrow I_i) \\ V_D(i, j-1) + \log P(D_i \rightarrow I_i) \end{cases}$$

$$V_D(i, j) = \max \begin{cases} V_M(i-1, j) + \log P(M_{i-1} \rightarrow D_i) \\ V_I(i-1, j) + \log P(I_{i-1} \rightarrow D_i) \\ V_D(i-1, j) + \log P(D_{i-1} \rightarrow D_i) \end{cases}$$

Όπου $P(X \rightarrow Y) = P(\text{Next State} = Y \mid \text{Current State} = X)$. Με άλλα λόγια η $P(X \rightarrow Y)$ είναι η πιθανότητα μετάβασης από την κατάσταση X στην Y . Παρατηρούμε επίσης τις log-πιθανότητες στους τύπους. Αυτό είναι μια προσθήκη που προήλθε από εμπειρικό συμπέρασμα. Χρησιμοποιώντας κανονικές πιθανότητες περιοριζόμαστε στο πεδίο τιμών $(0, 1]$ (το 0 ανοιχτό λόγω της προσθετικής εξομάλυνσης), γεγονός που δίνει ένα πολύ μικρό εύρος τιμών για τα τελικά alignment scores. Με την εφαρμογή του λογαρίθμου στο $(0, 1]$, γίνεται $(-\infty, 0]$ και έτσι ακόμα και μικρές διαφορές στις πιθανότητες τονίζονται πολύ περισσότερο. Για παράδειγμα παίρνουμε τα ίδια δεδομένα και τα περνάμε από τον αλγόριθμο Viterbi με log-πιθανότητες και με κανονικές πιθανότητες:

```
"summary": {
  "dataset": {
    "count": 40,
    "mean": 2.388487349935784,
    "best": 2.5159922961540815,
    "worst": 2.2723244184995135
  },
  "random": {
    "count": 40,
    "mean": 2.3625394785768745,
    "best": 2.5112982234733736,
    "worst": 2.2243126654164485
  }
}
```

Εικόνα 4: Μέσα alignment scores με χρήση μη-λογαριθμικών πιθανοτήτων

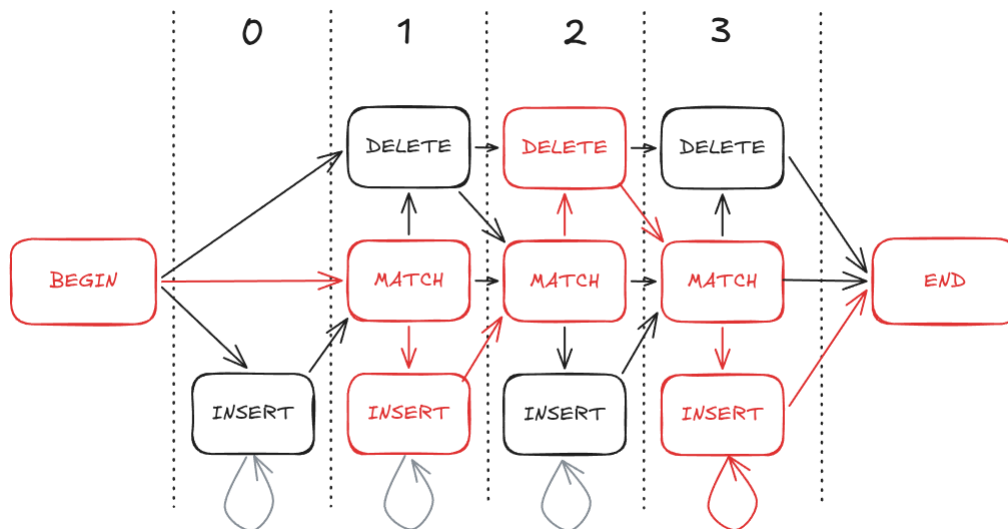
```
"summary": {
  "dataset": {
    "count": 40,
    "mean": -0.6640371244978545,
    "best": -0.3795551929931221,
    "worst": -1.0530276320756948
  },
  "random": {
    "count": 40,
    "mean": -1.9599256568353602,
    "best": -1.459060047230172,
    "worst": -2.276919842562983
  }
}
```

Εικόνα 5: Μέσα alignment scores με χρήση λογαριθμικών πιθανοτήτων

Βλέπουμε ότι με τις log-πιθανότητες, τα σκορ είναι αρνητικά. Παρόλο που δεν είναι όσο αισθητικό όσο τα θετικά σκορ, τα κρατάμε όπως έχουν. Τα σκορ υπολογισμένα με log-πιθανότητες έχουν μια επιπλέον επιθυμητή ιδιότητα: Το καλύτερο σκορ που μπορεί να υπάρξει είναι το 0. Επομένως, όσο πιο κοντά στο μηδέν, τόσο καλύτερα.

Όπως και στη καθολική στοίχιση, παράλληλα με το πίνακα DP V , κρατάμε και ένα πίνακα T ο οποίος περιέχει τις βέλτιστες μεταβάσεις για κάθε

$(i \in [0, L], j \in [0, N], s \in \{\text{INSERT}, \text{MATCH}, \text{DELETE}\})$. Όταν υπολογίσουμε όλα τα σκορ για κάθε (i, j, s) , τότε μπορούμε να ανακατασκευάσουμε το **βέλτιστο μονοπάτι**.



Εικόνα 6: Παράδειγμα βέλτιστου μονοπατιού Viterbi (το μονοπάτι είναι οι κόκκινες ακμές/κόμβοι)

Εκπαίδευση

Στην εκπαίδευση του profile HMM, “συντονίζουμε” τις κατανομές πιθανοτήτων εκπομπών και μεταβάσεων. Η διαδικασία είναι πολύ απλή και λιτή. Ως είσοδο, δέχεται ένα σύνολο N ακολουθιών $Y = \{y_1, y_2, \dots, y_N\}$. Για κάθε ακολουθία y_k εκτελεί πάνω της τον αλγόριθμο Viterbi και παίρνει το βέλτιστο μονοπάτι. Ύστερα διατρέχει το μονοπάτι και σε κάθε βήμα:

1. Προσμετράει την μετάβαση που προέκυψε από τη προηγούμενη κατάσταση S_{n-1} στην τωρινή S_n . ($S_{n-1} \rightarrow S_n$).
2. Προσμετράει την τωρινή κατάσταση αν είναι INSERT ή MATCH.
3. Τερματίζει την επανάληψη αν η κατάσταση είναι END.

Μετά την επανάληψη, έχουμε τρεις μετρητές: μετρητή μεταβάσεων, μετρητή καταστάσεων MATCH, και μετρητή καταστάσεων INSERT. Τέλος, οι μετρητές μετατρέπονται σε πιθανότητες με την ίδια διαδικασία που γίνεται και στην αρχικοποίηση (προσθετική εξομάλυνση).

Η εκπαίδευση του προφίλ βελτιώνει (refines) τις κατανομές πιθανοτήτων και κάνει το μοντέλο πιο ανθεκτικό σε διάφορους τύπους εισόδων στο βήμα της βαθμολόγησης (alignment scoring). Με την εκπαίδευση μπορεί πιο καθαρά να διακρίνει μεταξύ σε ακολουθίες της ίδιας οικογένειας και άσχετες ακολουθίες. Αυτό φαίνεται και στα αποτελέσματα που θα σχολιάσουμε στο τέταρτο μέρος.

IV – Βαθμολογία Στοίχισης (Alignment Scoring)

Στο τέταρτο και τελευταίο μέρος της εργασίας θα υλοποιήσουμε ένα αλγόριθμο που βαθμολογεί μια ακολουθία ως προς ένα εκπαιδευμένο profile HMM. Καλές βαθμολογίες θα δηλώνουν ότι η ακολουθία-είσοδος ταιριάζει με αυτές τις οποίες εκπαιδεύτηκε το profile HMM. Το αντίθετο θα σημαίνει ότι η ακολουθία είναι άσχετη, μερικώς ή ολικώς.

Ο αλγόριθμος που θα υλοποιήσουμε ονομάζεται **Forward Algorithm** και έχει πολύ κοντινή σχέση με τον αλγόριθμο Viterbi. Οι μόνες διαφορές είναι στον υπολογισμό του αναδρομικού τύπου και στο ότι ο forward algorithm δεν ασχολείται με μονοπάτια, άρα δεν χρησιμοποιεί πίνακα trace. Επομένως, ο forward algorithm έχει πίνακα DP $\mathbf{F} \in \mathbb{R}^{L \times N \times 3}$, ο οποίος έχει ίδιο σχήμα και φιλοσοφία με τον πίνακα $\mathbf{V}$ του Viterbi.

Όπως και στον Viterbi, ο αναδρομικός τύπος $F_s(i, j)$ εξαρτάται από την κατάσταση s και έτσι έχουμε πάλι τρεις ξεχωριστούς τύπους. Ωστόσο, δεν θα προσπαθήσω να τους διατυπώσω με μαθηματικά σύμβολα διότι είναι αρκετά πυκνός (αλλά όχι δυσνόητος) ο τύπος $F_s(i, j)$ για κάθε s . Αντί αυτού θα εξηγήσω λεκτικά τον υπολογισμό της κάθε τιμής $F_s(i, j)$. Ο forward algorithm δουλεύει πάνω στο ίδιο profile HMM με τον Viterbi, συνεπώς έχει το ίδιο HMM γράφημα με τις ίδιες ακμές και κόμβους. Αυτό σημαίνει ότι ο αναδρομικός τύπος θα υπολογίζει ακριβώς τις ίδιες παρελθοντικές τιμές για κάθε (i, j, s) . Επομένως στο μόνο που διαφέρει ο $F_s(i, j)$ από τον $V_s(i, j)$ του Viterbi είναι η συναθροιστική συνάρτηση (aggregation function). Ο $V_s(i, j)$ αξιοποιεί την $\max$ ενώ ο $F_s(i, j)$ υπολογίζει το άθροισμα των “προγόνων” του. Πιο συγκεκριμένα, υπολογίζει το LogSumExp ή RealSoftMax LSE (<https://en.wikipedia.org/wiki/LogSumExp>) διότι εργαζόμαστε σε λογαριθμικό χώρο (βλ. log-πιθανότητες στο Viterbi).

Η επιλογή του αθροίσματος προσδιορίζει το “any-path” κομμάτι της βαθμολογίας. Συγκριτικά, ο αλγόριθμος Viterbi είναι “best-path” διότι υπολογίζει το $\max$.

Δοκιμές και Αποτελέσματα

Εδώ θα σχολιάσουμε την απόδοση του forward algorithm για ακολουθίες που είναι επιλεγμένες από το datasetC, και έχουν ίδια μορφή με τις ακολουθίες του datasetA και datasetB η οποίες καθορίζουν το profile HMM. Επιπλέον, θα δούμε την απόδοση του για ακολουθίες που έχουν δημιουργηθεί τυχαία. Προτού αναλύσουμε τα νούμερα, αξίζει να δούμε πως δημιουργούνται οι τυχαίες ακολουθίες διότι δεν είναι εντελώς τυχαίως ο τρόπος.

Οι τυχαίες ακολουθίες δημιουργούνται με βάση ένα dataset “αναφορά”. Από αυτό το dataset εξάγουμε στατιστικά για το μήκος των επιμέρους ακολουθιών, συγκεκριμένα τη μέση τιμή (mean) μ και τη τυπική απόκλιση (standard deviation) σ . Με βάση αυτά τα στατιστικά, ορίζουμε μια Gaussian κατανομή $X \sim N(\mu, \sigma)$ από την οποία θα πηγάζουμε τυχαία τα μήκη των παραγόμενων τυχαίων ακολουθιών. Η κατανομή για την επιλογή των συμβόλων είναι ομοιόμορφη και δεν γίνεται καμία υπόθεση για αυτήν.

Ακολουθία	Mutation	Prefix/ Suffix	N	S	$\frac{S}{N}$
CAGATTAACCTTGCAAACCTGGCAACATTCAGTAT	1	3	33	-12.354	-0.37
TAGATTAACCTTTGCAAAGGCAACATTCAGTG	3	2	31	-15.199	-0.49
TAAGATTAACCTTTGCAAACCTGGCACATCAGTT	2	3	32	-14.834	-0.46
TCAGATTACTTGCAAACCTGGCAACATCAGTTC	3	4	32	-16.061	-0.50
GAGATAACTATGCAAACCTGGAACATTCATTA	3-4	2-3	31	-20.446	-0.66

Πίνακας 13: Ανάλυση βαθμολογιών στοίχισης για ακολουθίες του datasetC. Αναγράφονται και το πλήθος των πιθανών μεταλλάξεων και προθεμάτων/επιθεμάτων

Ακολουθία	N	S	$\frac{S}{N}$
CGTTGATTAACGCTTGCACTAACTTTCGTGTCAC	34	-51.116	-1.50
AGTACCATCCTTCAGTCCCTCGCTTGGTTAGAC	33	-69.044	-2.09
CTCGCCCGCTCCTGCCTTACCGTAATCAACA	31	-70.629	-2.27
ACCTGCCGAAGGCCGGGTATCGAGACCCTAC	31	-74.846	-2.41
GCTGATCGAAATGAGGGCGTCAACCCTGAGC	31	-65.513	-2.11

Πίνακας 14: Ανάλυση βαθμολογιών στοίχισης για τις τυχαίες ακολουθίες .

Από τους Πίνακες 13, 14 βλέπουμε ότι το μοντέλο πράγματι διακρίνει ακολουθίες που έχουν συγκεκριμένη δομή με τις τυχαίες. Οι παραπάνω πίνακες είναι μικρά δείγματα της ολικής δοκιμής πάνω σε ολόκληρο το datasetC και 40 τυχαίες ακολουθίες. Τα συνολικά στατιστικά αναγράφονται παρακάτω:

datasetC	# ακολουθίες	40
	Μέσο Score	-0.6650
	Καλύτερο Score	-0.3743
	Χειρότερο Score	-1.0864
Τυχαίες	# ακολουθίες	40
	Μέσο Score	-2.0507
	Καλύτερο Score	-1.5034
	Χειρότερο Score	-2.4143

Παρατηρούμε πόσο καλύτερα είναι τα alignment scores για όλες τις ακολουθίες του datasetC έναντι των τυχαίων. Συμπερασματικά, το πρόγραμμα δίνει ακριβώς τα αποτελέσματα που περιμέναμε να δούμε.

Παράρτημα

Το πρόγραμμα και οι λειτουργίες του (σύνθεση, MSA, profile HMM, και alignment scores) εκτελούνται μέσω γραμμής τερματικού με μια απλή διεπαφή (CLI). Το αρχείο `main.py` είναι αυτό που πρέπει να εκτελέσετε. Για κάθε εντολή, μπορείτε να προσθέσετε το flag `-help` για να εκτυπωθεί ένας κατάλογος επιλογών και εξηγήσεων.

```
➔ src git:(main) X python main.py --help
usage: main.py [-h] {synthesize,msa,profile,score} ...

positional arguments:
  {synthesize,msa,profile,score}
    synthesize          Generate datasets of ATGC sequences that are derived from specific
                        patterns.
    msa                 Perform multiple sequence alignment on datasetA, which is generated
                        from the synthesize sub-program.
    profile             Generate a profile HMM (Hidden Markov Model) initially based upon the
                        multiple sequence alignment. Then train the profile on datasetB.
    score              Calculate alignment scores for sequences using the any-path method.
                        Results are output to a JSON file.

options:
  -h, --help            show this help message and exit
```

Εικόνα 7: Κλήση `--help` στην `main.py`

Προκειμένου να τρέξετε το πρόγραμμα από σύνθεση μέχρι alignment scores, τρέξτε με τα υποπρογράμματα `synthesize`, `msa`, `profile`, `score` με αυτή τη σειρά.

```
➔ src git:(main) X python main.py synthesize --help
usage: main.py synthesize [-h] [--cache CACHE] [--seed SEED] -p PATTERNS

options:
  -h, --help            show this help message and exit
  --cache CACHE          The path of the cache directory
  --seed SEED           Seed the random number generator used for synthesis
  -p, --patterns PATTERNS
                        The text file containing the ATGC patterns to use. Each pattern should
                        be in a new line
```

```
➔ src git:(main) X python main.py msa --help
usage: main.py msa [-h] [--cache CACHE] --id ID

options:
  -h, --help            show this help message and exit
  --cache CACHE          The path of the cache directory
  --id ID               The student ID to be used in determine the  $\alpha$  parameter (used in the global alignment penalties)
```

```
→ src git:(main) X python main.py profile --help
usage: main.py profile [-h] [--cache CACHE] [--save-paths | --no-save-paths]

options:
  -h, --help            show this help message and exit
  --cache CACHE          The path of the cache directory
  --save-paths, --no-save-paths
                        Whether to store the Viterbi best paths on disk.
```

```
→ src git:(main) X python main.py score --help
usage: main.py score [-h] [--cache CACHE] [--seed SEED]

options:
  -h, --help            show this help message and exit
  --cache CACHE          The path of the cache directory
  --seed SEED           Seed the random number generator used for synthesis
```

Τα αποτελέσματα/έξοδοι κάθε υποπρογράμματος αποθηκεύονται στο δίσκο σας στο κατάλογο **_cache** ή στον κατάλογο που θα ορίσετε εσείς μέσω του flag `-cache`. Σε κάθε περίπτωση, το σύστημα έχει σύστημα logging και θα καταγράφεται η κάθε αποθήκευση αρχείων στο τερματικό σας.